

# BOOTCAMP MLOps



# BOOTCAMP MLOps:

## What we will learn to day:
## All You Need To Know About Docker

Danh Pham

# Goals

- Understand limitations of current approach.
- Understand Docker solution and core concepts.
- Know how to write a Dockerfile and run it.
- Know how to run multiple Dockerfiles with docker-compose.

# Contents


The Problem of Environment Inconsistency

The Docker Solution & Core Concepts

The Dockerfile - Infrastructure as Code

Docker Networking

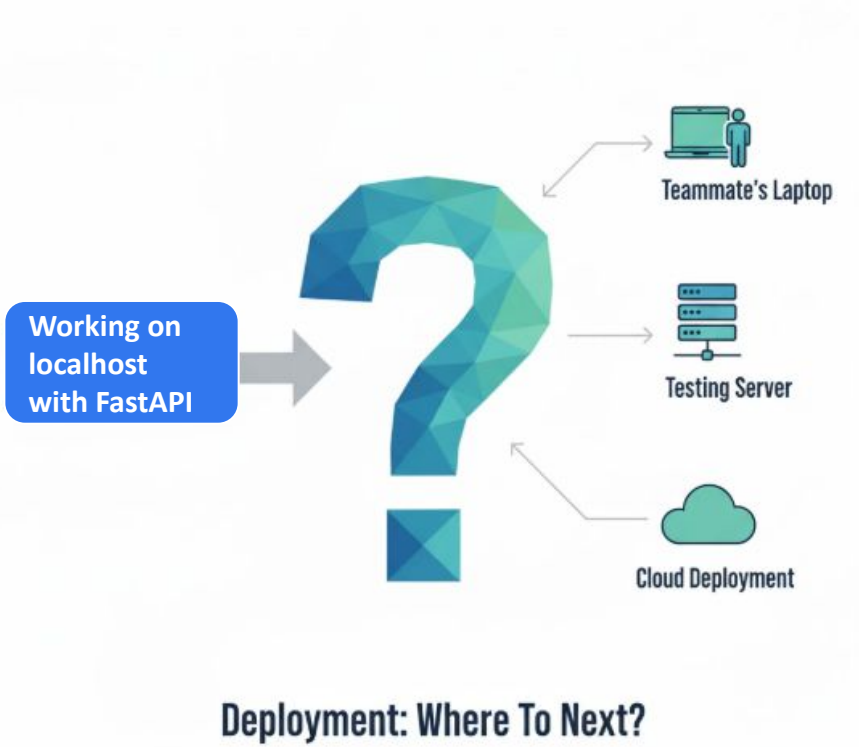
Docker Compose - Simple Orchestration

Hands-on Lab


COLE

# Where We Are & The Question We Must Answer

Recap:
Our ML API works on our machine. How do we guarantee it runs identically for others or in production?



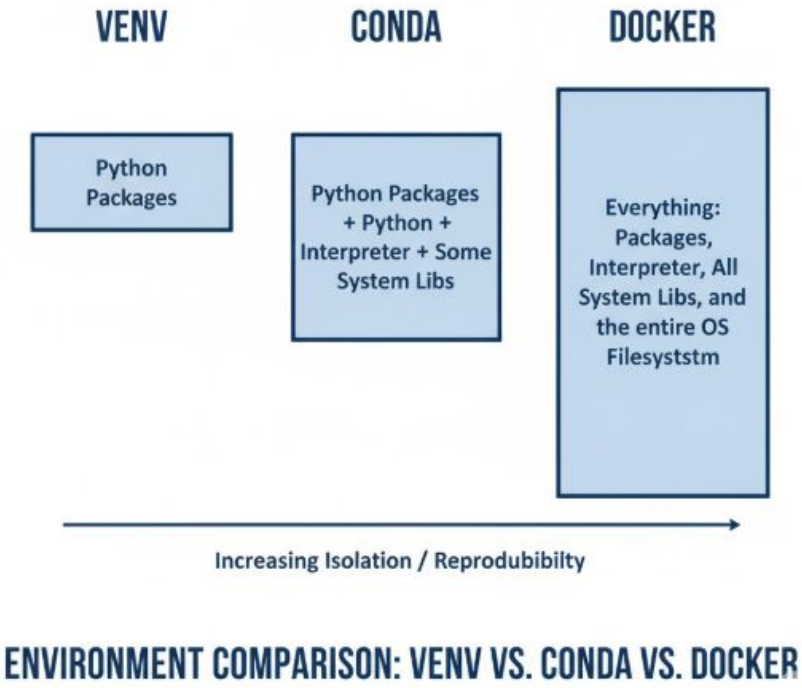
# The Environment Iceberg

- An application's true environment is mostly invisible.
- We see the code, but beneath the surface lie critical dependencies.
- Any difference in these hidden layers can cause catastrophic failures.



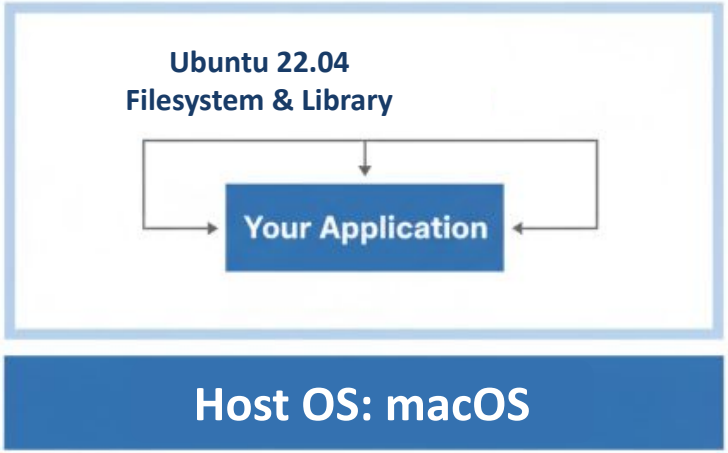
# Why venv and conda Aren't Enough

- venv/pip: Manages Python packages for a specific Python install.
- conda: Manages packages, Python versions, and some non-Python dependencies.
- The Gap: Neither controls the OS, kernel, or full system libraries.



# Docker's Solution: Package the Entire OS

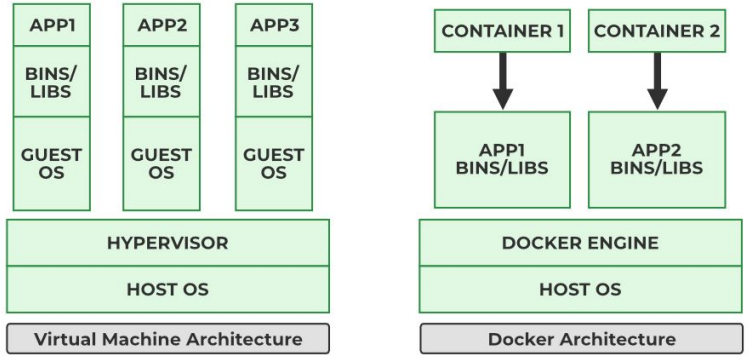
- Instead of managing dependencies inside an OS, Docker packages the OS user space with the application.
- This includes the filesystem, system libraries, and all binaries.
- The result is a self-contained "world" for your application to live in




Containerization: Running Anywhere

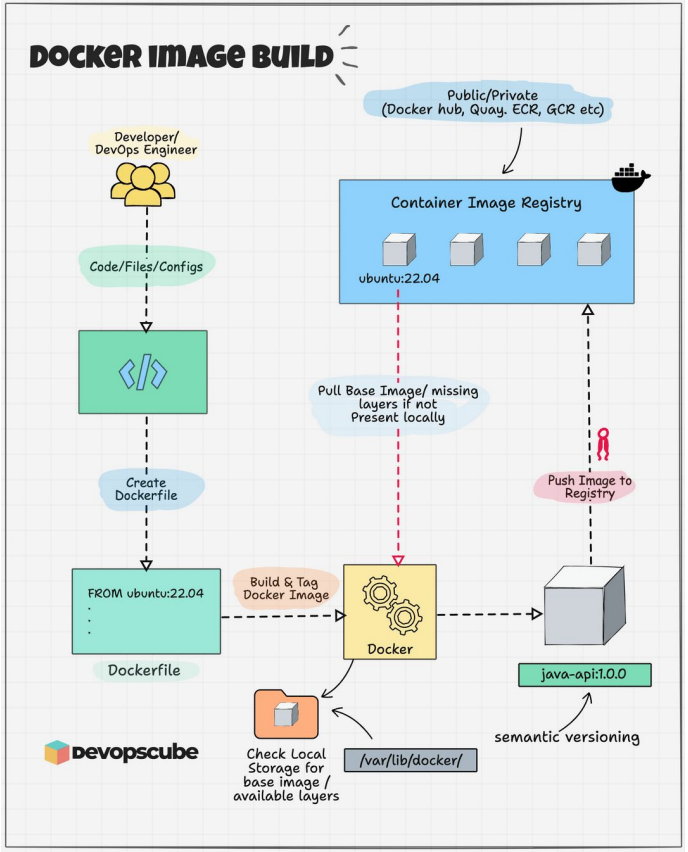
# Efficiency vs. Emulation: Containers vs. VMs

- Virtual Machines: Emulate hardware. Heavy, slow, and resource-intensive.
- Containers: Virtualize the OS. Lightweight, fast, and share the host kernel.
- The Result: You can run significantly more containers than VMs on the same hardware, leading to massive efficiency gains.



# The Docker Workflow: Recipe, Mix, Cake

- Dockerfile: A text file with instructions. The recipe for your environment.
- Image: A read-only, layered template built from the Dockerfile. The blueprint.
- Container: A runnable, live instance of an image. The running building.



# The Dockerfile: An Executable Blueprint

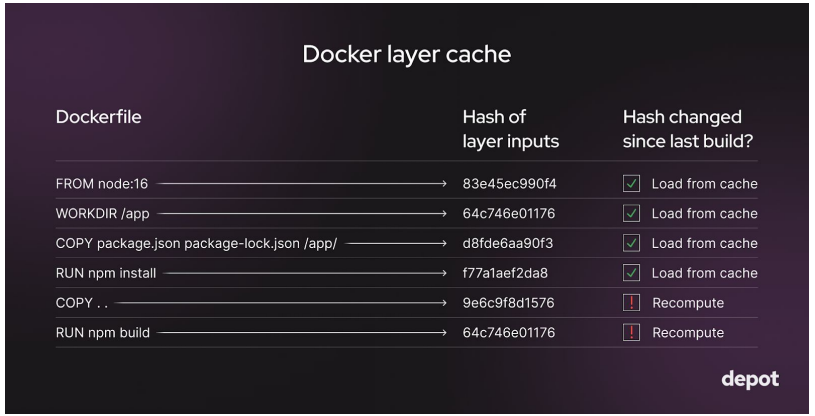
- FROM: Base image. Required.
- WORKDIR: Sets working directory.
- COPY: Copies host files to image.
- RUN: Executes build commands (e.g., pip install).
- EXPOSE: Documents listening port.
- CMD: Default container start command.

```
1   FROM python:3.9.1
2
3   RUN apt-get install wget
4
5   COPY ./requirements.txt ./
6   RUN pip install -r requirements.txt
7   RUN pip install psycopg2 pyarrow fastparquet
8
9   WORKDIR /app
10  COPY ingest_data.py ingest_data.py
11
12  ENTRYPOINT [ "python", "ingest_data.py" ]
13
14
```

COLE

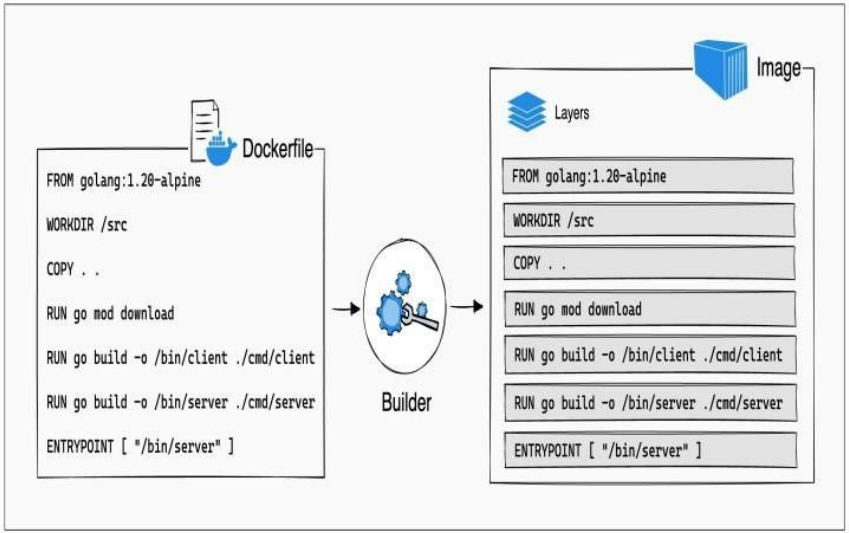
# How docker build Works: The Power of Cache

- Docker build runs Dockerfile instructions sequentially.
- It reuses cached layers for identical instructions, speeding up rebuilds.
- The cache is invalidated on the first changed command.



# Inside an Image: A System of Layers

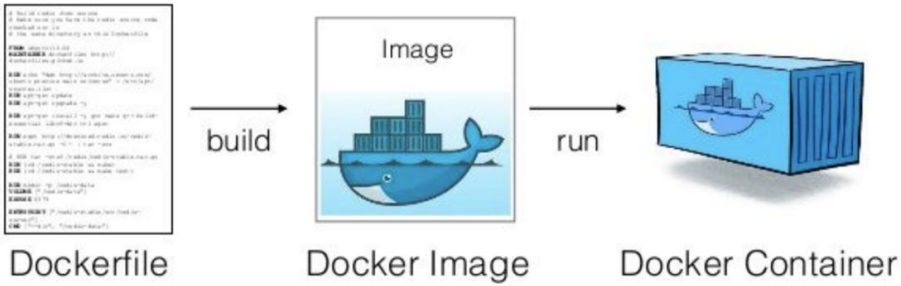
- Images are not monolithic; they are built from a series of read-only layers.
- Each instruction in a Dockerfile creates a new layer.
- Layers are cached and can be reused across different images.
- **This makes builds fast and storage efficient.**



# What is a Container, Really?

A container is a running host process with two properties:

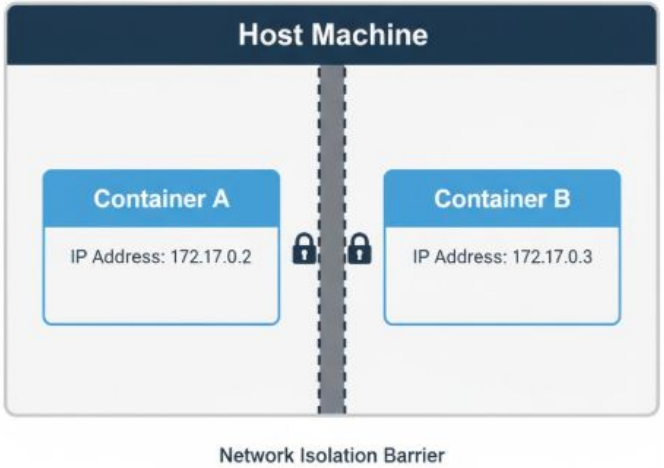
- Isolation: Private filesystem, network, and process tree via image layers.
- Writable Layer: A thin, writable layer atop read-only image layers for file changes.




COLE

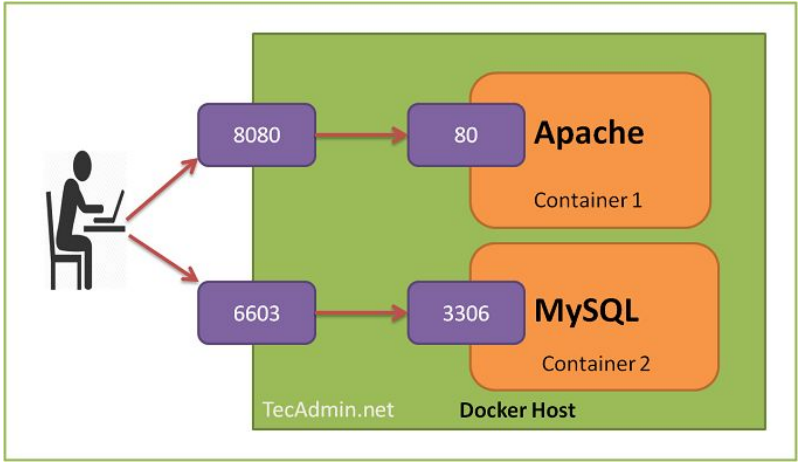
# Container Networking: Isolated by Default

- For security, every Docker container is attached to a private virtual network.
- Each container gets its own IP address and network stack.
- By default, containers cannot see or communicate with each other, even if they are on the same machine.



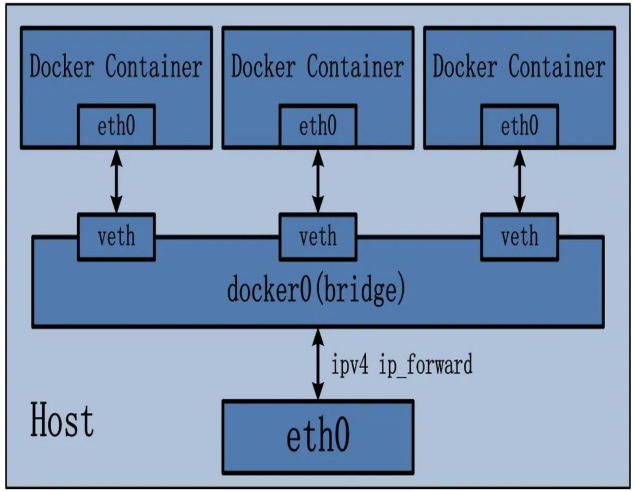
# Bridging the Gap: Port Mapping

- Port mapping (or "publishing") creates a bridge from a port on the host machine to a port inside the container.
- Syntax: docker run -p <HOST_PORT>:<CONTAINER_PORT>
- This allows external traffic to reach the application running inside its isolated network.



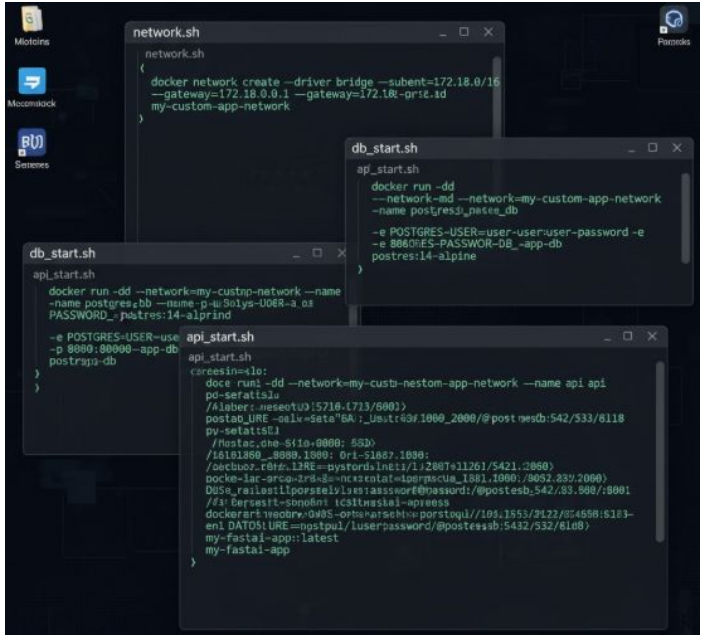
# User-Defined Bridge Networks

- **The Problem**: How does our API container find our Database container? Their IPs can change.
- **The Solution**: Create a custom **user-defined bridge network**.
- Attach multiple containers to the same network.
- Docker provides a built-in DNS service on these networks, allowing containers to find each other simply by their **name.**



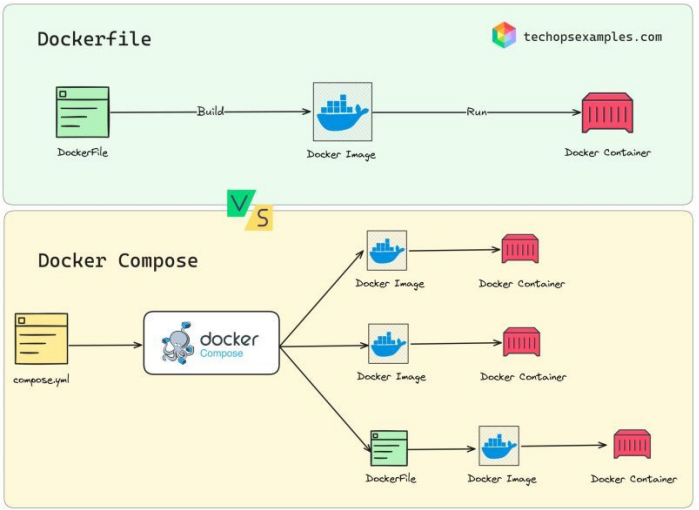
# The Pain of Manual Management

- Manually managing a multi-container application involves:
  - Creating networks.
  - Starting containers in the right order.
  - Typing long, error-prone docker run commands with all the correct flags.
  - Manually cleaning everything up.
- This is not scalable or reproducible.



# Docker Compose: Describing Your Application

- Docker Compose is a tool for **defining and running multi-container Docker applications.**
- You use a YAML file (docker-compose.yml) to configure your application's services, networks, and volumes.
- It turns a complex setup into a single, declarative configuration file.



# Inside docker-compose.yml

```
version: '3.8'

services:
  api:
    build: . # Build from local Dockerfile
    ports:
      - "8000:8000"
    networks:
      - app-network

  db:
    image: postgres:13 # Pull from Docker Hub
    environment:
      - POSTGRES_PASSWORD=mysecretpassword
    networks:
      - app-network

networks:
  app-network:
```

```yaml
version: '3.8' # Specifies the Docker Compose file format version

services:
  web: # Defines a service named 'web'
    build: . # Builds the image from the current directory (where a Docke
    ports:
      - "80:80" # Maps host port 80 to container port 80
    depends_on:
      - db # Ensures the 'db' service starts before 'web'
    environment:
      DATABASE_URL: postgres://user:password@db:5432/mydatabase # Example

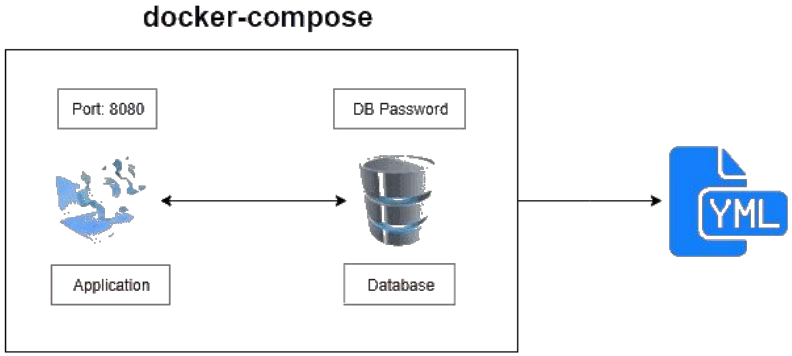
  db: # Defines a service named 'db'
    image: postgres:13 # Uses the official PostgreSQL 13 image from Docke
    environment:
      POSTGRES_DB: mydatabase # Sets the database name
      POSTGRES_USER: user # Sets the database user
      POSTGRES_PASSWORD: password # Sets the database password
    volumes:
      - db_data:/var/lib/postgresql/data # Mounts a named volume for pers

volumes:
  db_data: # Defines the named volume 'db_data'
```

COLE

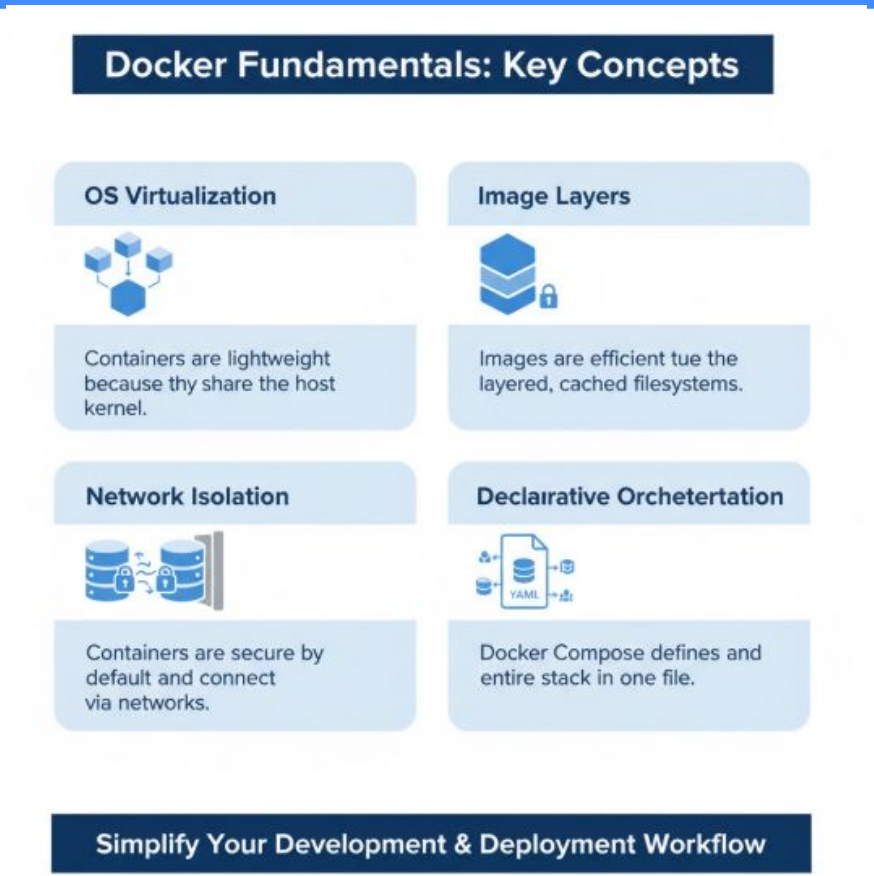
# One Command to Rule Them All

- docker-compose up: Reads the YAML file, builds images, creates networks, and starts all containers in the correct order.
- docker-compose down: Stops and removes all containers, networks, and other resources created by up.
- **This is the ultimate workflow for local development.**




COLE

# Key Theoretical Concepts



# Your mission

1. **Install Docker Desktop**: Ensure Docker Desktop is installed and running before the next lab session.
2. **Create a Docker Hub Account:** Sign up for a free account to push images later.
3. **Review:** Examine Dockerfile and docker-compose.yml examples; understand how each line defines the environment.



# Q&A



# Thank you for listening!

